

DOCUMENTO MAESTRO DEFINITIVO V1.8: SISTEMA SAO BAR 2026

1. RESUMEN EJECUTIVO Y ALCANCE TÉCNICO

El sistema SAO Bar 2026 se define como una infraestructura web multi-terminal de alto rendimiento diseñada para la gestión operativa y administrativa de jornadas nocturnas. Su propósito central es garantizar la integridad transaccional y el control de inventario en un entorno de alta demanda, eliminando las discrepancias de los registros manuales.

El propósito por el cual se desarrolla este proyecto es para probar cómo implementar un stack tecnológico robusto utilizando Next.js 16, Supabase, Tailwind CSS 4 y Server Actions, optimizando el diseño, el flujo de usuario y garantizando la consistencia de datos a nivel de base de datos a través de una colaboración estrecha con asistentes de IA agénticos.

Concepto de Jornada Nocturna No-Calendario

Dada la naturaleza operativa del establecimiento, el sistema ignora la linealidad de la fecha calendario del servidor. La actividad que inicia un viernes por la noche y concluye la madrugada del sábado se agrupa bajo un identificador único de "Jornada". Esto permite consolidar métricas financieras y de stock sin la fragmentación que causaría un cambio de fecha a midnight.

Entorno Operativo y Concurrencia

El sistema está dimensionado para la operación simultánea de 4 terminales activas bajo un modelo de datos centralizado:

- 3 Terminales de Comandas:** Dispositivos móviles o laptops destinados a la toma de pedidos en salón y barra por parte de los mozos.
- 1 Terminal de Administración:** Estación central (macOS/Windows) para el monitoreo de caja, auditoría física y cierre de jornada.

Stack Tecnológico

- Framework:** Next.js 16 (App Router) para una gestión de rutas y estados optimizada.
- Base de Datos:** Supabase (PostgreSQL) para la persistencia relacional con integridad ACID.
- Lógica de Servidor:** Server Actions para el procesamiento de reglas de negocio seguras.
- Estilos:** TailwindCSS 4 para un diseño ligero, responsivo y de alta velocidad de renderizado.

2. ARQUITECTURA DE DATOS Y CONCURRENCIA

Para mitigar riesgos de sobreventa y colisiones de datos en un entorno de 4 terminales, la arquitectura traslada la lógica crítica al motor de la base de datos.

Lógica de Stock y Bloqueo Pesimista

El sistema implementa el Procedimiento Almacenado (RPC) `procesar_comanda()`. Este procedimiento ejecuta una transacción atómica que utiliza la cláusula `FOR UPDATE` (Bloqueo Pesimista). Al recibir un pedido, el sistema realiza una pasada preliminar sobre las filas de los productos o insumos origen involucrados bloqueándolas y validando la disponibilidad de stock *antes* de realizar la inserción de la cabecera en la tabla `Comandas`. De esta manera, si el stock es insuficiente, se ejecuta un Rollback inmediato sin alterar ni consumir números de la secuencia

autoincremental (`numero_ticket`) del ticket de barra. Si todas las validaciones son correctas, el sistema descuenta las existencias, inserta el registro de cabecera consumiendo el número de ticket y graba los ítems de venta.

Lógica de Insumos y Stock Compartido (1:1)

Para optimizar el control de productos elaborados en el momento a partir de un recurso común y unitario (ej. pizzas de diversas variedades elaboradas a partir de una cantidad limitada de bollos de masa), el sistema soporta la asociación de stock compartido:

- **Enlace Relacional:** Un producto vendible (Pizza Muzza, Pizza Jamón) puede tener asociado un `insumo_compartido_id` apuntando al insumo origen (Bollo Pizza).
- **Deducción de Inventario:** Al venderse un producto enlazado, la deducción física del stock actual ocurre de forma automática y atómica sobre el insumo común en la base de datos.
- **Blindaje de Terminal contra Sobreventas:** El frontend realiza una acumulación en caliente de todos los ítems agregados al carrito de la terminal que compartan el mismo recurso. Si la sumatoria total del recurso iguala o supera su stock disponible, el sistema deshabilita las tarjetas de todas las variedades correspondientes e impide incrementar cantidades en el carrito local, bloqueando transacciones inválidas en caliente.

Sincronización Realtime

Se utiliza Supabase Realtime (WebSockets) agregando las tablas `Comandas` y `Productos` a la publicación `supabase_realtime`. Cuando el `stockActual` cambia en la base de datos tras una venta o reposición, o se inserta una nueva comanda, todas las terminales reflejan el nuevo estado de los badges de stock y del listado de historial en tiempo real sin necesidad de refrescar el navegador.

3. LÓGICA DE SEGURIDAD REFORZADA Y ROLES

Jerarquía de Roles

- **Rol Admin:** Acceso total, incluyendo Auditoría, Historial, ABM de Productos y Cierre de Caja.
- **Rol Empleado:** Acceso limitado exclusivamente a la toma de comandas y consulta de stock disponible.

JornadaGuard y Protección de Terminales (Bloqueo Realtime)

El sistema implementa el middleware `JornadaGuard` y un sistema de WebSockets para bloquear la navegación y funcionalidad de las terminales si no existe una jornada marcada como "Abierta". Esto evita registros accidentales fuera del horario operativo. Si el administrador cambia el estado de la jornada a `en_auditoria` o `cerrada` desde su terminal, la pantalla de los mozos es cubierta de inmediato por un banner opaco bloqueando la interfaz en tiempo real.

Flujo de Cierre y Seguridad Criptográfica

La transición de una jornada de 'Cerrada' a 'Abierta' y de 'Abierta' a 'En Auditoría' requiere la validación obligatoria del PIN de Administrador, el cual se procesa mediante un Hash SHA-256 en el cliente, asegurando que la clave nunca viaje en texto plano por la red.

Seguridad a Nivel de Fila (RLS)

El sistema delega la autorización de acceso directamente en el motor de la base de datos de Supabase (PostgreSQL) para evitar adulteraciones de datos que esquiven el frontend de Next.js:

- **Prevención de bucles recursivos:** Se define la función auxiliar `public.es_admin()` con la directiva `SECURITY DEFINER` en PostgreSQL. Esta función consulta el rol `'Admin'` del usuario logueado en la tabla `Usuarios` eludiendo de forma segura las restricciones de RLS de la propia tabla, lo cual evita el error de bucle recursivo infinito.
- **Acceso Administrativo Exclusivo:** Las tablas `Gastos` y `Auditoria_Inventario` se encuentran completamente bloqueadas para mozos (el rol de empleado ordinario no tiene permitido leerlas ni editarlas); toda operación (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) requiere que el usuario posea el rol `'Admin'`.

- **Integridad del Historial de Comandas:** Las tablas `Comandas` y `Comanda_Items` permiten la inserción de registros (`INSERT`) a todos los usuarios autenticados para que los mozos envíen comandas, verificando que el `usuario_id` del ticket coincida estrictamente con su sesión (`auth.uid() = usuario_id`). Sin embargo, se prohíbe la modificación (`UPDATE`) y el borrado (`DELETE`) a mozos ordinarios; estas capacidades de saneamiento histórico se reservan en exclusiva para administradores.
- **Control de Menú y Catálogos:** Las tablas `Productos`, `Categorias_Productos`, `Categorias_Gastos` y `Jornadas` admiten lectura libre (`SELECT`) para todos los autenticados para inicializar la terminal de ventas, pero restringen la creación o edición únicamente a usuarios administradores.
- **Control de Perfiles:** La tabla `Usuarios` permite que los empleados lean perfiles generales, pero bloquea la manipulación de PINs o altas de personal de forma exclusiva a administradores.

4. DICCIONARIO DE DATOS RELACIONAL

A continuación, se detalla la estructura relacional obligatoria para asegurar la integridad referencial.

Entidad: Usuarios

- `id`: UUID (Primary Key). Mapeado directamente a la tabla interna de usuarios de Supabase Auth (`auth.users`) mediante FK.
- `email`: String (Único).
- `nombre`: String.
- `rol`: Enum ('Admin', 'Empleado').
- `pin`: Hash SHA-256 de 64 caracteres. Utilizado únicamente para validación de PIN en terminales de seguridad del administrador.
- `activo`: Boolean. Default `true`.

Entidad: Categorias_Productos

- `id`: UUID (Primary Key).
- `nombre`: String (Único).
- `color_fondo`: String (Formato hexadecimal de 7 caracteres, ej. `#FF0000`. Validado por CHECK regex `^[0-9A-Fa-f]{6}$`).
- `color_texto`: String (Formato hexadecimal de 7 caracteres. Validado por CHECK regex `^[0-9A-Fa-f]{6}$`).
- `activo`: Boolean. Default `true`.

Entidad: Categorias_Gastos

- `id`: UUID (Primary Key).
- `nombre`: String (Único, ej. *Insumos, Limpieza, Servicios, Otros, Personal, Mantenimiento*).
- `activo`: Boolean. Default `true`.

Entidad: Productos

- `id`: UUID (Primary Key).
- `nombre`: String.
- `categoria_id`: Foreign Key (`Categorías_Productos`) ON DELETE RESTRICT.
- `precio`: Numeric(10,2).
- `stockIdeal`: Integer (Referencia visual semanal).
- `stockInicial`: Integer (Actualizado solo en reposición o creación).
- `stockActual`: Integer. Afectado por `procesar_comanda()` únicamente si `controla_stock` es `true`.
- `unidad`: String (u, lt, ml).

- **activo**: Boolean. Default **true** (Maneja la baja lógica del catálogo).
- **vendible**: Boolean (Determina si se muestra en el menú de comandas de los mozos. False equivale a insumo interno).
- **controla_stock**: Boolean (Determina si el producto realiza seguimiento y descuento dinámico de existencias).
- **insumo_compartido_id**: UUID (Opcional). FK autorreferencial (**Productos**) ON DELETE SET NULL. Apoya la relación de stock compartido (pizza -> bollo).

Entidad: Jornadas

- **jornada_id**: UUID (Primary Key).
- **estado**: Enum ('abierta', 'en_auditoria', 'cerrada').
- *Constraint de Unicidad Parcial*: **CREATE UNIQUE INDEX unique_active_jornada ON Jornadas (estado) WHERE estado IN ('abierta', 'en_auditoria');**. Garantiza que no exista más de una jornada activa o en revisión a la vez.
- **fecha_inicio**: Timestamp con zona horaria.
- **fecha_fin**: Timestamp con zona horaria.
- **total_efectivo**: Numeric(10,2).
- **total_mp_lista**: Numeric(10,2).
- **total_mp_real**: Numeric(10,2).
- **comision_mp**: Numeric(10,2) (**total_mp_lista - total_mp_real**).
- **total_general**: Numeric(10,2) (**total_efectivo + total_mp_real**).
- **gastos_totales**: Numeric(10,2).
- **ganancia_neta**: Numeric(10,2) (**total_efectivo + total_mp_real - gastos_totales**).

Entidad: Comandas

- **comanda_id**: UUID (Primary Key).
- **jornada_id**: Foreign Key (**Jornadas**).
- **usuario_id**: Foreign Key (**Usuarios**).
- **numero_ticket**: SERIAL (Incremental autogestionado por la base de datos).
- **nro_beeper**: Integer. Opcional (Constraint: valor numérico entero del 1 al 20 inclusive).
- **fecha**: Timestamp con zona horaria.
- **total**: Numeric(10,2).
- **medio_pago**: Enum ('Efectivo', 'Mercado Pago', 'Regalo').

Entidad: Comanda_Items (N:N Break)

- **id**: UUID (Primary Key).
- **comanda_id**: Foreign Key (**Comandas**) ON DELETE CASCADE.
- **producto_id**: Foreign Key (**Productos**).
- **cantidad**: Integer.
- **precio_unitario_historico**: Numeric(10,2) (Snapshot del precio al momento de la venta).

Entidad: Gastos

- **id**: UUID (Primary Key).
- **jornada_id**: Foreign Key (**Jornadas**).
- **categoria_id**: Foreign Key (**Categorías_Gastos**).
- **descripcion**: String/Text.
- **monto**: Numeric(10,2).

Entidad: Auditoria_Inventario

- `id`: UUID (Primary Key).
- `jornada_id`: Foreign Key (`Jornadas`).
- `producto_id`: Foreign Key (`Productos`).
- `conteo_fisico`: Integer. Ingresado por el Admin en el Cierre de Caja; solo aplica para productos con `controla_stock = true`.
- `unidades_utilizadas`: Integer. Unidades vendidas o insumos consumidos.
- `unidades_regaladas`: Integer. Unidades entregadas mediante comandas con medio de pago 'Regalo'.
- `stock_inicial`: Integer. Guarda la foto del stock de apertura de cada producto al iniciar la jornada. Resuelve inconsistencias financieras en el historial de desvíos.

5. MÓDULO DE REPORTES Y ANALÍTICA

Generación Dinámica y Auditoría Anti-Robo

Los reportes se generan mediante JOINs relacionales en tiempo real. El motor de auditoría aplica la siguiente fórmula de conciliación de stock para productos con `controla_stock = true`:

- **Stock Teórico = Stock Inicial - Unidades Vendidas - Unidades Regaladas** (Leídos directamente de la foto en `Auditoria_Inventario`).
- **Desvío = Conteo Físico - Stock Teórico.**

Visualización de Desvíos y Ordenamiento

- **Token Error:** Si el Conteo Físico difiere del Stock Teórico (para productos con seguimiento de stock), la interfaz resalta el registro con el color funcional Error (`#E2484A`).
- **Jerarquía de Datos:** Los reportes de stock se ordenan descendientemente por volumen de ventas, priorizando los productos de mayor rotación.

Conciliación de Mercado Pago

Se realiza el cotejo financiero de Mercado Pago en el cierre:

- `comision_mp = total_mp_lista - total_mp_real`.
- **Token Advertencia:** Si `comision_mp` supera el 10% del `total_mp_lista`, se resalta en color Advertencia (`#BA7517`).

Módulo de Rendimiento y Rotación de Ventas

- **Consolidación en Memoria:** El sistema agrupa relacionamente en tiempo real todas las comandas de la jornada por `producto_id`, sumando las unidades vendidas e ingresos brutos por cada artículo.
- **Exclusión de Insumos:** Este reporte filtra de forma automática únicamente los productos marcados como vendibles (`vendible = true`), eliminando insumos operativos para mantener la claridad analítica.
- **Exclusión de Regalos:** Las comandas con medio de pago 'Regalo' se omiten del cálculo de ventas comerciales de esta pestaña para evitar falsear las métricas de volumen cobrado y total general de facturación por producto.
- **Visualización de Rendimiento:** Se presenta en el historial una pestaña dedicada ("Rendimiento") que lista todos los productos vendidos ordenados descendientemente por rotación (unidades vendidas) para evaluar de forma directa el volumen e ingresos del menú de la noche.

6. SISTEMA DE TOKENS VISUALES (SIN TABLAS)

Tipografía (Google Fonts)

- **Creepster:** Estilo Display para Logotipo "SAO Bar" y elementos decorativos de alta identidad.
- **Livvic:** Estilo UI para toda la interfaz (botones, tablas, inputs, lecturas, reportes, historial).

Colores de Marca

- **Negro:** #080A0D (Fondo primario).
- **Naranja:** #F26A1B (Acciones primarias y paneles).
- **Naranja Alternativo:** #F25922 (Interacciones y estados activos).
- **Cyan:** #30CFF2 (Énfasis de navegación y logo).
- **Blanco:** #F2F2F2 (Texto principal y contrastes).

Colores Funcionales

- **Efectivo:** #1D9E75 (Confirmación y éxito).
- **Mercado Pago:** #378ADD (Pagos digitales).
- **Regalo / Cortesía:** #7C3AED (Regalos de la casa).
- **Error/Peligro:** #E2484A (Desvíos, alertas y errores).
- **Advertencia:** #BA7517 (Alerta de stock bajo o comisiones fuera de lo normal).
- **Gris UI:** #9D9D9D (Textos secundarios y deshabilitados).

7. ESPECIFICACIONES DE INTERFAZ

Layout General (Shell) y Menú Activo

- **Topnav (56px) / AdminNav:** Se utiliza el componente interactivo de cliente `<AdminNav />` para la navegación administrativa. Identifica mediante `usePathname` la pestaña activa (Productos, Comandas, Caja, Cierre, Historial, Calculadora) y le asigna color blanco #F2F2F2 y un borde inferior cian #30CFF2. Oculto en impresión (`print:hidden`).

Módulo ABM de Productos (Patrón Master-Detail)

- **Fondo y Tarjeta Central:** Posee un fondo de pantalla naranja corporativo (`bg-[#F26A1B]`) con todo el menú administrativo flotando dentro de una gran tarjeta oscura central (`bg-[#1A1A1A]`) de bordes redondeados y sombras pronunciadas.
- **Grilla de Productos:** Muestra el catálogo categorizado. Los productos con `activo = false` se muestran al 40% de opacidad.
- **Barra de Filtros:** Incluye búsqueda en vivo por texto, selector por categoría, selector por clasificación de ítem («Insumos», «Productos (Vendibles Cerrados)», «Elaboración Instantánea», «Todos los Items») y un checkbox reactivo para elegir si listar o no productos inactivos/dados de baja.
- **Formulario de Edición/Alta de Producto:**
 - Toggle de clasificación: «Producto para la venta» (`vendible = true`) vs. «Insumos» (`vendible = false`).
 - Checkbox de inventario: «Seguimiento de stock» (`controla_stock = true`). Si el tipo es *Insumo*, el checkbox se fuerza a `true` y se deshabilita.
 - Asociación de Stock Compartido: Si se enlaza un insumo origen, los campos de Stock Inicial, Stock Actual y Stock Ideal se ocultan en el modal, mostrando una nota cian descriptiva. En la persistencia se graban silenciosamente en `0`.
 - El campo `stockActual` es de solo lectura durante la visualización general, pero permite ingresos de reabastecimiento directo si se edita.
- **Edición y Creación de Categorías:**
 - Modal flotante para dar altas y bajas lógicas.
 - Los colores de fondo y texto de la categoría son configurados por el administrador y validados estrictamente bajo expresión regular hexadecimal en el cliente.

- Regla de consistencia: No se puede activar o guardar un producto vendible si su categoría asociada se encuentra desactivada (`activo = false`).

Terminal de Comandas

- **Sidebar Izquierdo (80px):** Selector de categorías vertical (solo categorías activas).
- **Grilla de Productos:** Muestra tarjetas de productos con `vendible = true`. Los badges de stock dinámico solo se calculan e interactúan si `controla_stock = true`. Al vender la última unidad, la interfaz descuenta el stock local de forma optimista e instantánea deshabilitando la tarjeta al microsegundo antes de realizar la recarga silenciosa de fondo.
- **Subdivisión por Categorías:** Al seleccionar la categoría "Todos los productos", la grilla clasifica y ordena dinámicamente las tarjetas mediante cabeceras divisionales y líneas de relieve coloreadas de acuerdo al tono de la categoría, separando visualmente las cervezas, comidas, tragos y cafetería.
- **Carrito y Medio de Pago:** Detalle del pedido, selector de beeper opcional (1 al 20) y selección de medio de pago (Efectivo/Mercado Pago/Regalo).
- **Botón de Regalo:** Se renderiza de forma compacta (`h-8`, bordes discontinuos transparentes y fuente pequeña de color negro `#080A0D` seminegrita cursiva) por debajo de los dos botones principales para evitar clics accidentales. Al seleccionarlo, se despliega una nota de advertencia explicativa en alto contraste.
- **Impresión de Comanda para Cocina:** Al registrar el pedido de manera exitosa, el modal de confirmación cambia su etiqueta visual a "Comanda Nro" y ofrece un botón de impresión de comanda (Cian `#30CFF2`) que activa el diálogo de impresión local. Este ticket se formatea en blanco y negro para ticketeras térmicas (76mm), ocultando la interfaz general mediante directivas CSS de impresión. Su diseño operativo contiene metadatos de fecha y hora local (con formato localizado en español, ej: "Viernes 28 de marzo de 2025"), número secuencial de la comanda en gigante (ej. #42), beeper asignado de forma condicional con icono 🚨 y el detalle de ítems de productos con cantidades prefijadas por x (sin precios ni importes de caja).
- **Scroll Adaptativo por Altura (Zoom/Pantallas Pequeñas):** Mide mediante listener de `resize` el alto útil (`window.innerHeight`). Si este es inferior a `780px` (por zoom de 150-175% o pantallas pequeñas), desactiva las alturas fijas y el bloqueo de desborde del contenedor raíz, permitiendo scroll general de navegador, y restringe el listado de pedidos a un máximo de `max-h-[350px]` para asegurar la visibilidad total de los controles de pago e historial.
- **Cierre de Sesión Seguro:** La acción de «Salir» (mozo y administrador) despliega un modal con fondo desenfocado (`backdrop-blur-sm`) que requiere confirmación explícita.

8. PROCEDIMIENTOS OPERATIVOS CRÍTICOS

Envío de Comanda (Transacción Atómica)

Al invocar `procesar_comanda()`:

1. Validación de estado de jornada (Debe ser 'Abierta').
2. Bloqueo de filas en productos o insumos compartidos vía `FOR UPDATE`.
3. Validación de disponibilidad (`stockActual >= cantidad`) — solo si `controla_stock = true``.
4. Validación del número de beeper (Si está presente, debe estar en el rango de 1 a 20 inclusive).
5. Inserción en comandas.
6. Inserción en `comanda_items` (Capturando precio actual).
7. Deducción del `stockActual` del producto o insumo asociado — solo si `controla_stock = true``.
8. Si `medio_pago = 'Regalo'`, el total de la cabecera es forzado a `0.00`.
9. Tras completarse la transacción atómica, la terminal mozo recibe de retorno el identificador correlativo (`numero_ticket` autoincremental de la base de datos) y la fecha oficial de registro de la comanda, habilitando la generación e impresión del ticket físico para la cocina.

Flujo de Cierre de Jornada

Este proceso debe ser ejecutado en orden estrictamente atómico:

1. **Validación de PIN:** Validación obligatoria mediante Hash SHA-256 en cliente.
 2. **Transición de Estado:** Cambio de jornada a 'En Auditoría', bloqueando comandas nuevas.
 3. **Carga de Gastos + Stock real MP:** Registro de egresos y conciliación manual de posnet de Mercado Pago (`total_mp_real`).
 4. **Auditoría Física:**
 - Carga de conteos en `Auditoria_Inventario` — únicamente para productos con `controla_stock = true``.
 - Persiste el conteo manual físico, las unidades utilizadas/vendidas, las unidades regaladas y el snapshot del `stock_inicial` de apertura.
 - **Tratamiento Inteligente de Insumos Compartidos:** Si un insumo posee dependientes asociados (ej. Bollo Pizza), el sistema lo clasifica como autocalculado (oculta el input manual de consumo y computa estáticamente la sumatoria total del consumo de las pizzas hijas en base a comandas), simplificando la tarea del administrador.
 5. **Consolidación Final:** El sistema calcula totales de efectivo, Mercado Pago, comisiones y ganancia neta, y transiciona la jornada a 'Cerrada'. Atómicamente, se actualiza el `stockActual` y `stockInicial` de todos los productos auditados en la tabla `Productos` con su correspondiente `conteo_fisico`, sirviendo este último como el stock de partida para la apertura de la siguiente jornada.
-

9. ESPECIFICACIÓN TÉCNICA: MÓDULO CALCULADORA DE COSTOS (STATELESS)

Resumen Funcional

La Calculadora de Costos permite a la administración simular costos de insumos de manera volátil (React State local en el cliente) sin impactar el inventario o finanzas oficiales en Supabase. Al refrescar la aplicación, los datos se eliminan.

Flujo en Pantalla

- **Grilla Libre:** Fila de inputs vacíos que permite al usuario registrar libremente el Nombre del Insumo, Cantidad y Precio Unitario, calculando de manera instantánea el subtotal e incrementando el Total General.
- **Autocompletado Sugerido:** Al escribir en el nombre del insumo, un dropdown dinámico consulta en caliente el catálogo real del servidor (`obtenerProductos`) para autocompletar rápidamente.
- **Aviso en Cursiva:** Muestra en pantalla el texto simple en cursiva gris: «Aviso: Esta calculadora es una herramienta de simulación.».
- **Interacciones:** Botón «+ Agregar insumo» para sumar filas al estado y botones «X» para remover filas.

Estilos de Impresión A4

Al presionar el botón «Guardar / Imprimir» (o mediante `Ctrl+P`), el navegador activa directivas `@media print`:

- El fondo naranja de pantalla y la tarjeta oscura se transforman en una hoja de reporte blanca.
 - Se oculta la barra de navegación lateral, el menú superior administrativo global, y todos los controles interactivos de edición (los botones de eliminar fila y agregar insumo).
 - Los inputs cambian a texto plano (sin bordes ni parches de formulario) en el PDF.
 - Se dibuja un membrete con el logotipo circular del bar `/images/sao-logo.png` con fondo blanco, título, subtítulo, fecha y hora de impresión.
 - Se renderiza al pie del reporte A4 una firma institucional gris: «Este documento es una simulación generada por la Calculadora de Costos de SAO Bar.»
-

10. CASOS DE USO DETALLADOS

Caso de Uso: Tomar comanda (CDU-SAO-001)

- **Actor/es:** Empleado (Mozo).
- **Precondiciones:** Jornada en estado 'abierta'. Productos seleccionados con `controla_stock = true` tienen `stockActual >= cantidad`. Beeper ingresado está en el rango de 1 a 20.
- **Flujo principal:**
 1. El mozo selecciona artículos del menú clasificados por categoría.
 2. El sistema actualiza optimistamente el stock en caliente deshabilitando temporalmente el botón si las unidades caen a 0.
 3. El mozo puede opcionalmente ingresar el número de beeper (1 al 20) y seleccionar el medio de pago, donde se incluye la opción de Regalo.
 4. Envía la comanda.
 5. Si el medio de pago es Regalo, el sistema notifica en pantalla la aclaración de cortesía y, al enviar la comanda, el procedimiento atómico `procesar_comanda` fuerza el total a 0.00 en la cabecera del ticket, registrando de igual forma la salida física del inventario para la auditoría de desvíos.
 6. Al confirmarse el pedido de forma exitosa, el sistema habilita un modal que muestra el número de ticket autoincremental asignado por la base de datos y el botón de Impresión de Comanda.
 7. El mozo presiona imprimir y el navegador despliega la ventana de impresión local con un diseño adaptado para comandas térmicas monocromáticas de cocina (sin importes, con cantidades antepuestas por la letra x, beeper visible y fecha formateada localmente en español).
- **Postcondiciones:** Se decrementa el stock en base de datos. Si la comanda es de regalo, se registra con valor nulo en caja comercial, pero con persistencia de cantidades físicas e historial para auditoría interna.

Caso de Uso: Apertura de jornada (CDU-SAO-002)

- **Actor/es:** Administrador.
- **Precondiciones:** La jornada anterior se encuentra en estado 'cerrada'. No existen jornadas activas.
- **Flujo principal:**
 1. El administrador ingresa el PIN correspondiente.
 2. Define el saldo inicial de caja en efectivo y Mercado Pago.
 3. Presiona abrir jornada.
- **Postcondiciones:** Transición a estado 'abierta', permitiendo la toma de comandas en todas las terminales.

Caso de Uso: Gestionar productos (CDU-SAO-003)

- **Actor/es:** Administrador.
- **Descripción:** Alta, modificación y baja lógica de productos del menú o insumos.
- **Flujo principal:**
 1. El administrador accede a `/admin/productos`.
 2. Define nombre, precio, categoría, tipo (Venta/Insumo) y seguimiento de stock.
 3. Al editar, se puede actualizar el stock por reabastecimiento directo.
- **Postcondiciones:** Actualización en base de datos. Si se da de baja un producto, pasa a `activo = false` ocultándose del menú pero preservando la integridad del historial.

Caso de Uso: Gestionar categorías (CDU-SAO-004)

- **Actor/es:** Administrador.
- **Descripción:** Gestión de categorías de productos y sus colores de visualización.
- **Flujo principal:**
 1. El administrador abre el modal de categorías.
 2. Carga el nombre y los colores en formato Hexadecimal (validado por regex).
- **Postcondiciones:** Si una categoría es desactivada, los productos vendibles pertenecientes a ella quedan bloqueados para su venta de forma automática.

Caso de Uso: Cierre de jornada (CDU-SAO-005)

- **Actor/es:** Administrador.
- **Precondiciones:** Jornada en estado 'abierta'.
- **Flujo principal:**
 1. Ingreso de PIN de Administrador (Validado localmente mediante hash SHA-256).
 2. Bloqueo transaccional de comandas (Jornada pasa a 'en_auditoria').
 3. Registro de egresos y el ingreso manual de posnet de Mercado Pago.
 4. En la sección de Auditoría Física de Inventario, el sistema clasifica de forma inteligente los productos. Si detecta un insumo con productos dependientes asociados (stock compartido), bloquea su edición de consumo manual y autocalcu la su gasto sumando el consumo real de los productos finales vendidos.
 5. El sistema recupera el total de unidades regaladas durante la jornada y las deduce del stock teórico en conjunto con las ventas antes de calcular el desvío físico definitivo.
- **Postcondiciones:** El sistema consolida la ganancia neta (**efectivo + MP real - gastos**), guarda la foto del stock de apertura y actualiza el stock actual de catálogo con los conteos físicos para servir de base para la próxima jornada.

Caso de Uso: Simulación de Costos (CDU-SAO-006)

- **Actor/es:** Administrador.
- **Descripción:** Simulación de compra o consumo en caliente de insumos y exportación en formato A4.
- **Flujo principal:**
 1. Ingresa a la calculadora.
 2. Carga filas de insumos con autocompletado y define cantidades e importes unitarios.
 3. Compara totales recalculados.
 4. Presiona Guardar / Imprimir.
- **Postcondiciones:** Se lanza el cuadro de diálogo de impresión del navegador renderizando el PDF A4 limpio en blanco, sin persistencia en base de datos.

Caso de Uso: Consultar Historial de Jornadas y Rendimiento (CDU-SAO-007)

- **Actor/es:** Administrador.
- **Precondiciones:** El usuario está autenticado con rol de Administrador. Existen jornadas en estado 'cerrada' registradas en el sistema.
- **Flujo principal:**
 1. El Administrador accede a la sección de Historial del panel de administración (**/admin/historial**).
 2. El sistema carga en memoria y renderiza el listado de jornadas cerradas de forma cronológica descendente, habilitando un filtro de fechas (Desde / Hasta) para acotar la búsqueda.
 3. El Administrador selecciona una jornada específica del panel lateral izquierdo.
 4. El sistema realiza la consulta a la base de datos y presenta la información de la jornada seleccionada distribuida en pestañas dinámicas:
 - **Resumen Financiero:** Consolida el flujo de caja (Efectivo inicial, Efectivo final, Mercado Pago en lista de comandas versus el monto ingresado manualmente por el Posnet real, deducción automática de comisiones y balance de gastos).
 - **Auditoría de Inventario:** Muestra la conciliación de stock para productos con seguimiento activo. Se contrasta el Stock Inicial de la jornada, las unidades vendidas, las unidades regaladas, el Stock Teórico resultante, el Conteo Físico final ingresado por el Administrador y el desvío. Los desvíos se resaltan en color de error si hay discrepancias.
 - **Gastos:** Listado con las descripciones, categorías y montos de los egresos registrados durante esa noche.
 - **Comandas:** Historial individual de todas las comandas creadas, mostrando el número de ticket correlativo oficial, hora de registro, total facturado y el medio de pago utilizado.
 - **Rendimiento:** Muestra una grilla con los productos vendidos ordenados de forma descendente por rotación (volumen de unidades), calculando ingresos brutos y excluyendo insumos operativos y comandas marcadas como regalo.
- **Postcondiciones:** El Administrador obtiene una vista detallada, inalterable y consolidada de los eventos financieros y de inventario de la jornada seleccionada.

Caso de Uso: Autenticación y Control de Acceso (CDU-SAO-008)

- **Actor/es:** Administrador / Empleado.
- **Precondiciones:** El usuario posee credenciales registradas en Supabase Auth y un perfil mapeado en la tabla **Usuarios**.
- **Flujo principal:**
 1. El usuario accede a la pantalla principal (/).
 2. Ingresa su correo electrónico y contraseña.
 3. El sistema valida las credenciales y recupera el perfil del usuario en la base de datos local.
 4. El sistema evalúa el estado del usuario: si está marcado como inactivo (**activo = false**), bloquea el acceso e interrumpe la sesión.
 5. Si el perfil es activo, el sistema redirige al usuario según su rol:
 - **Empleado:** Redirección forzada al área de toma de comandas (/comandas).
 - **Administrador:** Acceso completo habilitado a las rutas de gestión administrativa (/admin/*).
- **Postcondiciones:** El usuario inicia sesión de forma segura y se aplican las restricciones de acceso a nivel de interfaz de usuario y políticas de base de datos (RLS).